

Rust, CUDA and Verus

A PoC on Rust-CUDA Verifiable Kernels

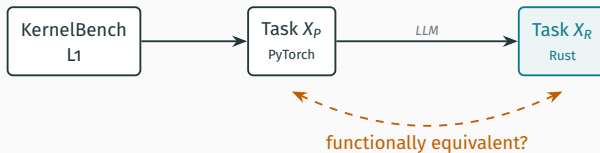
Mahadi Hasan Anamul Haque Imtiaz

September 26, 2026

UIUC SRSE 2026

From PyTorch to Rust

Source of the CUDA kernels: KernelBench Level 1 tasks (for now).



Goal. Guarantee that the translated task computes the same function as the original.

$$X_P, X_R : \mathcal{X} \rightarrow \mathcal{Y} \quad \forall x \in \mathcal{X}. \llbracket X_P \rrbracket(x) = \llbracket X_R \rrbracket(x)$$

Two KernelBench L1 tasks

19_ReLU.py

```
class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.relu(x)

batch_size = 4096
dim = 393216
def get_inputs():
    x = torch.rand(batch_size, dim)
    return [x]
def get_init_inputs():
    return []
```

What it computes. Replaces every negative entry with zero; the shape is unchanged.

$$x \in \mathbb{R}^{4096 \times 393216}, \quad y_{ij} = \max(0, x_{ij})$$

47_Sum_reduction_over_a_dimension.py

```
class Model(nn.Module):
    def __init__(self, dim: int):
        super(Model, self).__init__()
        self.dim = dim
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.sum(x, dim=self.dim, keepdim=True)

batch_size = 128
dim1 = 4096
dim2 = 4095
reduce_dim = 1
def get_inputs():
    x = torch.rand(batch_size, dim1, dim2)
    return [x]
def get_init_inputs():
    return [reduce_dim]
```

What it computes. Adds up the entries along the middle axis, keeping it as size 1.

$$x \in \mathbb{R}^{128 \times 4096 \times 4095}, \quad y_{b,o,k} = \sum_{j=0}^{4095} x_{b,j,k}$$

Source: KernelBench @ 423217d, KernelBench/level1/; docstrings and imports omitted.

Faithful spec extraction

Initially, Mahadi worked on how to *faithfully* extract a specification from a PyTorch source. The Python file is the specification, so the job is **denotation**: what function does it compute?

Task 47 (PyTorch)

```
return torch.sum(x, dim=1, keepdim=True)
```

Question about I/O

- 1 How is the input x held?
- 2 Which inputs feed output cell r?
- 3 What is done with them?
- 4 What type is an output value?
- 5 How many outputs, and what is each?

Spec it creates

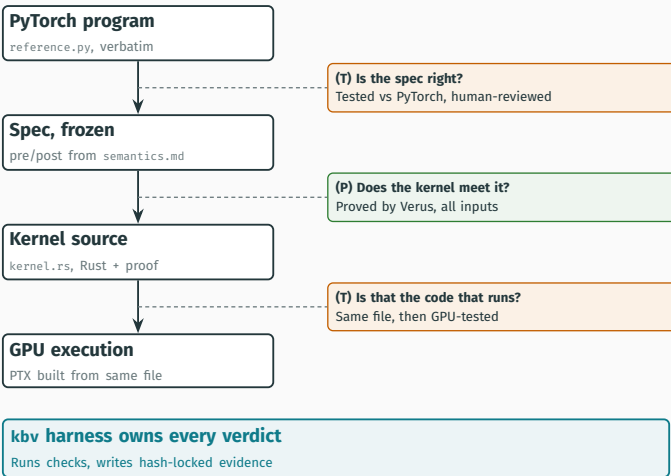
```
⇒ x: Seq<i32>, rows: int, cols: int // flat, row-major
⇒ row(x, r, cols) = x.subrange(r*cols, r*cols + cols)
⇒ fold_sum(s) = if s.len() == 0 { 0 } else { s[0] + fold_sum(s.skip(1)) }
⇒ fold_sum(s: Seq<i32>) -> int // overflow cannot hide in the spec
⇒ torch_sum_dim1(x, rows, cols) = Seq::new(rows, |r| fold_sum(row(x, r, cols)))
```

Insights. Questions about I/O reveal the necessary specs. However, faithfulness up to this point remains a spec-testing problem; human audit remains the final standard.

Verus-KernelBench

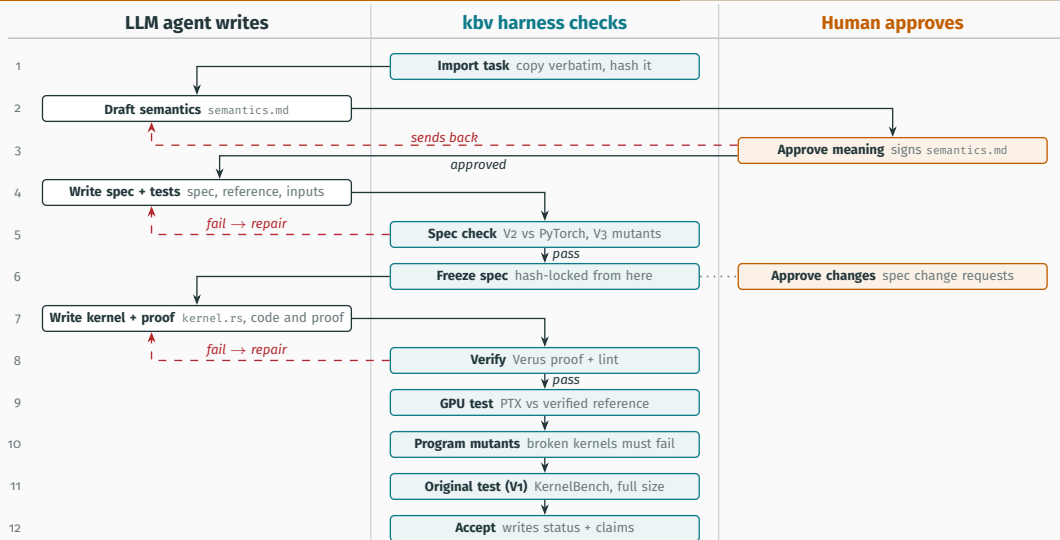
Anamul expanded this idea to a proof-of-concept in the GitHub repo **Verus-KernelBench**.

Most of the steps can be automated, although **unfaithfully**.

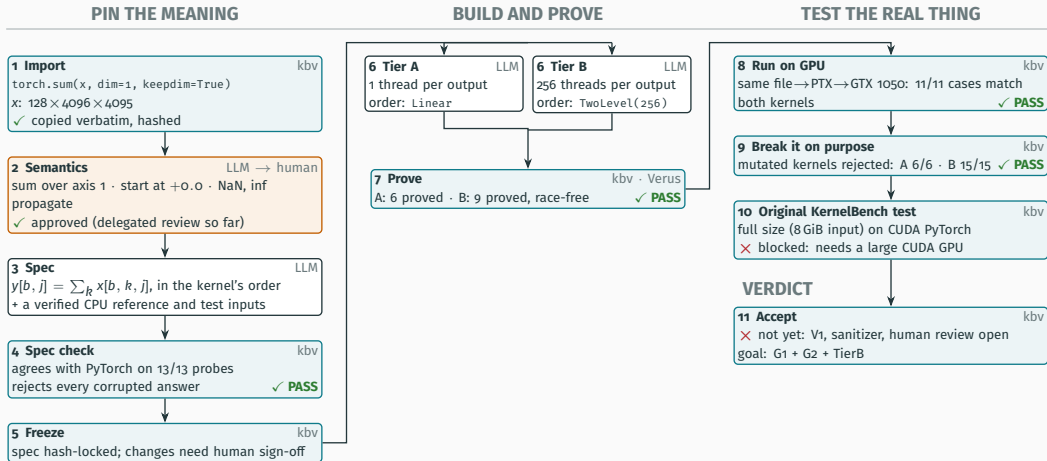


(T) = checked by testing (P) = checked by proof

A verbose diagram of the PoC



Task 47 through the PoC



Review on the quality of LLM-generated kernels

We have agreed that the quality we can maintain on this benchmark dataset depends on the time we have. Currently we found two potential deadlines:

Target. LLM4Code @ ICSE 2027, 13 Nov (tentative)

Ambitious. ISSTA 2027, 11 Jan: propose a study/methodology with dataset

Criteria	Confidence
Spec faithfulness	High
Technical rigor	Low (Nov) Mid (Jan)

Technical rigor is what Mahadi thinks the current dataset lacks, for example:

- Verus does not support float arithmetic, but Mahadi finds that Anamul's current PoC replaces float addition with a trusted `#[verifier::external_body]` function.
- SOTA papers like *Kuiper* do not do this; rather, they show the approximation explicitly.
- The dataset's Rust code is generated by LLMs without the frameworks real GPU programmers use, i.e. `cuda-oxide` or `cuTile-Rust`.
- In Mahadi's own findings, using `cuda-oxide` or `cuTile-Rust` with in-place Verus annotations, which real GPU communities would actually use, is more time-consuming and less likely to fit the Nov deadline.

Blockers, Literature and Potential

Blockers

- Compiled CUDA code (PTX) needs an Nvidia GPU to run and test.
- Anamul has a GTX 1050 with 2 GB VRAM, which is insufficient for many KernelBench L1 tasks.
- Mahadi has a Mac and currently works with the Google Colab CLI, which is also insufficient.
- **Ask:** access to a modern Nvidia GPU with 24 GB VRAM, like an RTX 4090.

Literature

- *Kuiper: GPU Kernel Verification in Pulse* does almost the same work we are doing, so their dataset is helpful to cross-check against our Verus port.
- *ProofWright: Towards Agentic Formal Verification of CUDA* claims it will open-source its artifact MLRocq, which converts PyTorch $\rightarrow$ Rocq “deterministically”, but has not yet. Early access would potentially make our work easier and give another base to cross-check against.

Potential

- The KernelBench L1 tasks are our choice of initial contribution.
- The Rust-CUDA space has much more potential for contributions, but I find that they are not feasible within the time and scope of this program.